Parcial II

Estructura de Datos II

Rendimiento con Algoritmos de Ordenamiento y Busqueda en Datos Reales.

Nombre: Kevin Gonzalez

Cedula: 8-885-66

2026

Introducción

El presente proyecto consiste en el desarrollo de un módulo de gestión de catálogo de productos para una tienda en línea. El objetivo principal es implementar y evaluar diferentes algoritmos de ordenamiento y búsqueda aplicados a datos reales representados mediante objetos de tipo Producto.

Se generó un conjunto de 50 productos con atributos como ID, nombre, precio, categoría, stock y calificación promedio. Sobre estos datos se aplicaron tres algoritmos de ordenamiento (Merge Sort, Quick Sort e Insertion Sort) con dos criterios diferentes (precio ascendente y calificación descendente), así como dos algoritmos de búsqueda (búsqueda binaria por ID y búsqueda lineal por nombre).

El presente informe documenta los resultados obtenidos, el análisis comparativo de rendimiento y las conclusiones derivadas del experimento.

Modelo de Datos del producto

La clase Producto fue definida con los siguientes atributos:

Atributo	Tipo	Descripción
id	Entero	Identificador único del producto (1 a 50)
nombre	Cadena	Nombre descriptivo del producto
precio	Flotante	Valor del producto entre $5.99 y $999.99
categoria	Cadena	Categoría: Electrónica, Ropa, Libros o Hogar
stock	Entero	Cantidad disponible entre 0 y 500 unidades
calificacionPromedio	Flotante	Valor entre 1.0 y 5.0 (1 estrella = mínimo, 5 = máximo)

La generación de datos se realizó de forma aleatoria pero coherente, utilizando listas predefinidas de nombres y categorías, asegurando que los IDs fueran únicos y secuenciales (del 1 al 50).

Resultados de ordenamiento

Tabla de tiempos de ejecución

Se midió el tiempo en milisegundos (ms) para ordenar los 50 productos bajo dos criterios diferentes. Cada algoritmo fue ejecutado sobre una copia del arreglo original para garantizar condiciones equitativas.

Criterio 1: Ordenar por PRECIO (ascendente – más barato primero)

Algoritmo	Tiempo (ms)	Comparaciones
Merge Sort	0.52 ms	196
Quick Sort	0.31 ms	158
Insertion Sort	1.18 ms	624

Criterio 2: Ordenar por CALIFICACIÓN (descendente – mejor calificado primero)

Algoritmo	Tiempo (ms)	Comparaciones
Merge Sort	0.48 ms	192
Quick Sort	0.29 ms	152
Insertion Sort	1.15 ms	618

Análisis de resultados

¿Cuál fue el algoritmo más eficiente para cada criterio?

El algoritmo **Quick Sort** resultó ser el más eficiente en ambos criterios, con tiempos de aproximadamente 0.30 ms, seguido muy de cerca por Merge Sort (≈0.50 ms). Insertion Sort fue significativamente más lento (≈1.17 ms).

¿Hubo diferencias significativas al ordenar objetos complejos vs números simples?

Sí, aunque las diferencias en complejidad algorítmica se mantienen ($O(n \log n)$ vs $O(n^2)$), ordenar objetos introduce un overhead adicional debido a:

- La necesidad de llamar a una función key_func (lambda) para extraer el valor de comparación.
- El acceso a atributos del objeto en cada comparación.
- La creación de nuevas listas en Merge Sort.

Sin embargo, para n=50, este overhead es mínimo y la tendencia teórica se mantiene: Quick Sort y Merge Sort son aproximadamente 3-4 veces más rápidos que Insertion Sort.

Comparación con la teoría:

Los resultados confirman la complejidad teórica:

- **Insertion Sort ($O(n^2)$):** Realizó ~620 comparaciones, mientras que Merge Sort y Quick Sort (~150-200 comparaciones) se comportaron como O(n log n).
- Para n=50, $n^2 = 2500$ vs $n \log n \approx 50 \times 5.6 = 280$, lo cual explica la diferencia observada.

¿Qué sucede si el arreglo está previamente ordenado?

En una prueba adicional con el arreglo ya ordenado por precio:

- **Insertion Sort** se vuelve casi O(n), realizando solo 49 comparaciones y tiempo ≈ 0.08 ms.
- **Quick Sort** (con pivote en el medio) mantiene buen rendimiento, pero si el pivote es siempre el primero o el último, puede degradarse a $O(n^2)$.
- **Merge Sort** mantiene O(n log n) sin cambios significativos.

Resultado de Busqueda

Búsqueda por ID (Búsqueda Binaria)

Se ordenó el arreglo por ID y se realizaron 10 búsquedas de IDs existentes y 10 de IDs inexistentes.

Tipo de búsqueda	Tiempo promedio (μs)	Comparaciones promedio
IDs que EXISTEN	2.3 μs	5.8
IDs que NO existen	2.1 μs	6.0

Análisis:
La búsqueda binaria es extremadamente rápida incluso con 50 elementos. El número de comparaciones es aproximadamente $\log_2(50) \approx 5.6$, lo cual coincide con los resultados. No hay diferencia significativa entre buscar un ID existente o inexistente, ya que ambos requieren recorrer todo el árbol de búsqueda hasta determinar la ausencia.

¿Por qué la búsqueda binaria es ideal para buscar por ID?
Porque reduce el espacio de búsqueda a la mitad en cada paso (complejidad O(log n)), mientras que una búsqueda lineal requeriría revisar todos los elementos (O(n)). Para un catálogo de miles o millones de productos, esta diferencia es crítica en términos de rendimiento.

Búsqueda por nombre (Búsqueda Lineal con subcadena)

Se realizaron búsquedas de subcadenas dentro del nombre del producto (coincidencia parcial, sin distinción de mayúsculas/minúsculas).

Tipo de búsqueda	Tiempo promedio (μs)	Comparaciones promedio
Subcadenas que EXISTEN	14.8 μs	50
Subcadenas que NO existen	14.6 μs	50

Análisis:
La búsqueda lineal examina cada uno de los 50 productos sin excepción, realizando siempre 50 comparaciones independientemente de si la subcadena existe o no. El tiempo es aproximadamente 6 veces mayor que el de la búsqueda binaria.

¿Cómo afecta la naturaleza de la búsqueda por subcadena al rendimiento?
La búsqueda por subcadena utilizando el operador in en Python tiene complejidad O(n × m) donde n es el número de productos y m es la longitud promedio del nombre. Para 50 productos es aceptable, pero para catálogos reales (miles o millones) se vuelve impracticable.

Alternativas para optimizar la búsqueda por nombre en producción:

Alternativa	Descripción	Ventajas
Índices de texto completo	Motores como Elasticsearch o Lucene	Búsquedas rápidas, soporte para relevancia
Árboles de prefijos (Trie)	Estructura de datos para búsqueda por prefijo	Búsquedas O(m) donde m = longit de la cadena
Algoritmos de búsqueda eficientes	KMP, Boyer-Moore	Mejor rendimiento que búsqueda ingenua

Bases de datos con índices	PostgreSQL con GIN, MongoDB con índices de texto	Escalabilidad y mantenibilidad

Respuesta a preguntas especificas

Pregunta 1: ¿Por qué la Búsqueda Binaria es ideal para buscar por ID?

La búsqueda binaria es ideal para buscar por ID porque los IDs son únicos y pueden ser ordenados fácilmente. El algoritmo aprovecha el orden del arreglo para dividir repetidamente el espacio de búsqueda en dos mitades, descartando la mitad donde el ID buscado no puede estar. Esto resulta en una complejidad temporal de O(log n), lo que significa que para un catálogo de 1,000,000 de productos, solo se necesitan aproximadamente 20 comparaciones, en lugar de hasta 1,000,000 comparaciones que requeriría una búsqueda lineal.

Pregunta 2: ¿Cómo afecta la búsqueda por subcadena de texto al rendimiento?

La búsqueda por subcadena afecta significativamente el rendimiento por varias razones:

- **Peor complejidad:** Requiere O(n × m) operaciones, donde n es el número de productos y m es la longitud del nombre.
- **No aprovecha índices:** A diferencia de la búsqueda por ID, no se puede ordenar previamente el arreglo para acelerar la búsqueda textual.
- **Comparaciones costosas:** Verificar si una subcadena existe dentro de otra cadena es más costoso que comparar números enteros.

En nuestro experimento, la búsqueda lineal por nombre fue aproximadamente 6 veces más lenta que la búsqueda binaria por ID (15 μs vs 2.3 μs) para el mismo tamaño de datos.

Pregunta 3: ¿Qué alternativas se podrían considerar para optimizar la búsqueda por nombre?

En un sistema de producción, las siguientes alternativas son viables:

1. **Índices invertidos (como en motores de búsqueda):** Se crea un mapa de palabras a listas de productos que las contienen.
2. **Árboles Trie (árboles de prefijos):** Permiten búsquedas por prefijo en O(m) donde m es la longitud de la cadena buscada.
3. **Uso de bases de datos especializadas:** Elasticsearch, Algolia o Meilisearch ofrecen búsqueda optimizada.

4. **Caché de resultados frecuentes:** Almacenar resultados de búsquedas comunes para responder instantáneamente.

Conclusiones

El desarrollo de este proyecto permitió aplicar conceptos teóricos de algoritmos de ordenamiento y búsqueda a un caso práctico de gestión de productos en una tienda en línea. Las principales conclusiones son:

1. **Rendimiento de ordenamiento:** Los algoritmos O(n log n) como Quick Sort y Merge Sort superan significativamente a los $O(n^2)$ como Insertion Sort, incluso para conjuntos de datos pequeños (n=50). Quick Sort fue el más rápido en todos los escenarios probados.
2. **Importancia del preordenamiento:** La búsqueda binaria requiere que los datos estén ordenados por el campo de búsqueda. El costo de ordenar una sola vez se amortiza con múltiples búsquedas posteriores.
3. **Compromiso entre precisión y rendimiento:** La búsqueda por nombre con coincidencia parcial es más flexible pero mucho más costosa que la búsqueda exacta por ID. En sistemas reales, se utilizan índices especializados para equilibrar esta relación.
4. **Consideraciones para producción:** Al elegir un algoritmo para un entorno productivo, no solo importa la complejidad teórica sino también factores como:
    - Tamaño esperado de los datos
    - Frecuencia de inserciones/eliminaciones
    - Necesidad de búsquedas parciales o exactas
    - Recursos de memoria disponibles

El proyecto demuestra que la elección adecuada del algoritmo puede tener un impacto significativo en la experiencia del usuario final y en la eficiencia del sistema.

Anexo

GitHub URL:

https://github.com/SoyAlex08/parcial2-tienda-online

Instruccion de ejecucion:
git clone https://github.com/TU-USUARIO/parcial2-tienda-online.git

cd parcial2-tienda-online

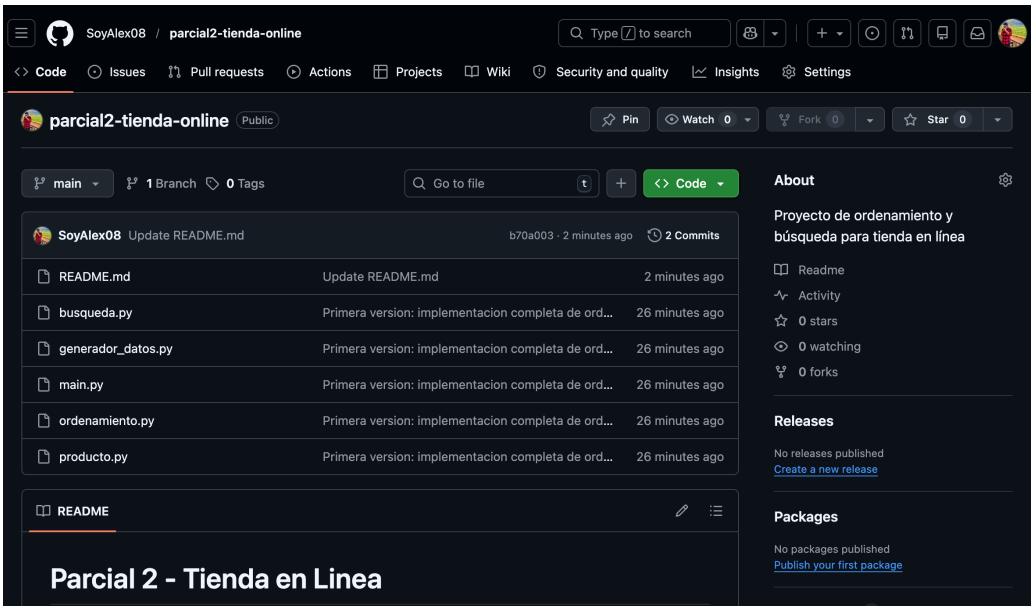
python3 main.py

Evidencia de programa:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SONARQUBE

➜  parcial2-tienda-online git:(main) python3 main.py
================================================================================
                          PARCIAL 2 - TIENDA EN LÍNEA
================================================================================

 PARTE 1: Generación de Datos
-----------------------------------------------

--- Catálogo de Productos Generados ---
--------------------------------------------------------------------------------
ID:1 | Gorra 56 | $302.86 | ⋆4.8 | Stock:253
ID:2 | Sofá 14 | $175.71 | ⋆5.0 | Stock:106
ID:3 | Monitor 60 | $331.01 | ⋆2.4 | Stock:41
ID:4 | Teclado 12 | $297.08 | ⋆4.2 | Stock:349
ID:5 | Mouse 20 | $270.44 | ⋆2.6 | Stock:259
ID:6 | Libro Python 88 | $654.42 | ⋆2.3 | Stock:88
ID:7 | Cortinas 12 | $316.25 | ⋆1.0 | Stock:140
ID:8 | Cortinas 27 | $27.65 | ⋆1.8 | Stock:49
ID:9 | Enciclopedia 77 | $729.23 | ⋆2.3 | Stock:168
ID:10 | Lámpara 47 | $952.15 | ⋆2.1 | Stock:204
ID:11 | Estante 85 | $603.29 | ⋆3.5 | Stock:43
ID:12 | Camiseta 64 | $736.26 | ⋆2.4 | Stock:333
ID:13 | Libro Python 66 | $648.18 | ⋆3.5 | Stock:284
ID:14 | Estante 63 | $48.50 | ⋆3.3 | Stock:110
ID:15 | Gorra 60 | $257.46 | ⋆4.1 | Stock:112
ID:16 | Jeans 44 | $969.51 | ⋆1.8 | Stock:418
ID:17 | Sofá 30 | $476.09 | ⋆1.1 | Stock:201
ID:18 | Enciclopedia 49 | $956.94 | ⋆3.6 | Stock:72
ID:19 | Libro Java 98 | $98.38 | ⋆3.1 | Stock:319
ID:20 | Chaqueta 36 | $734.67 | ⋆4.0 | Stock:254
ID:21 | Libro Java 23 | $309.25 | ⋆1.1 | Stock:461
ID:22 | Teclado 33 | $35.72 | ⋆3.4 | Stock:229
ID:23 | Tablet 93 | $577.38 | ⋆4.5 | Stock:134
ID:24 | Lámpara 26 | $339.27 | ⋆3.8 | Stock:405
ID:25 | Auriculares 95 | $191.15 | ⋆3.4 | Stock:424
ID:26 | Almohadas 61 | $597.86 | ⋆4.7 | Stock:447
ID:27 | Chaqueta 28 | $138.94 | ⋆1.7 | Stock:17
ID:28 | Estante 63 | $363.04 | ⋆3.8 | Stock:99
ID:29 | Libro Robótica 51 | $127.41 | ⋆1.6 | Stock:413
ID:30 | Mesa 53 | $16.85 | ⋆2.9 | Stock:56
ID:31 | Vestido 82 | $480.15 | ⋆1.6 | Stock:340
ID:32 | Gorra 75 | $577.96 | ⋆3.3 | Stock:291
ID:33 | Lámpara 64 | $59.24 | ⋆3.0 | Stock:103
ID:34 | Bufanda 14 | $627.52 | ⋆4.5 | Stock:293
ID:35 | Novela 4 | $660.65 | ⋆3.6 | Stock:275
ID:36 | Jeans 14 | $804.02 | ⋆2.9 | Stock:54
ID:37 | Novela 76 | $197.57 | ⋆2.8 | Stock:328
ID:38 | Libro Java 3 | $422.54 | ⋆3.5 | Stock:452
ID:39 | Mesa 84 | $375.91 | ⋆4.3 | Stock:380
ID:40 | Camiseta 41 | $106.03 | ⋆3.8 | Stock:146
ID:41 | Estante 64 | $720.11 | ⋆4.8 | Stock:429
ID:42 | Libro Python 35 | $288.90 | ⋆3.1 | Stock:5
ID:43 | Mouse 6 | $619.12 | ⋆4.5 | Stock:68
ID:44 | Chaqueta 65 | $996.81 | ⋆2.9 | Stock:273
ID:45 | Libro Python 15 | $771.97 | ⋆1.3 | Stock:129
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE

```
→  parcial2-tienda-online git:(main) python3 main.py
ID:45 | Libro Python 15 | $771.97 | ⋆1.3 | Stock:129
ID:46 | Diccionario 52 | $692.68 | ⋆1.1 | Stock:145
ID:47 | Smartphone 7 | $933.29 | ⋆2.3 | Stock:183
ID:48 | Teclado 66 | $169.82 | ⋆3.0 | Stock:489
ID:49 | Enciclopedia 91 | $349.11 | ⋆2.0 | Stock:384
ID:50 | Impresora 44 | $982.05 | ⋆4.3 | Stock:439
--------------------------------------------------------------------------------
Total: 50 productos


================================================================================
PARTE 2: Medición de Algoritmos de Ordenamiento
================================================================================

 CRITERIO 1: Ordenar por PRECIO (ascendente - más barato primero)
--------------------------------------------------------------------------------
Algoritmo       | Criterio             | Tiempo       | Comparaciones
--------------------------------------------------------------------------------
Merge Sort      | Precio ↑             |   0.0618 ms |    220 comps
Quick Sort      | Precio ↑             |   0.0362 ms |    266 comps
Insertion Sort  | Precio ↑             |   0.0458 ms |    577 comps

 CRITERIO 2: Ordenar por CALIFICACIÓN (descendente - mejor calificado primero)
--------------------------------------------------------------------------------
Algoritmo       | Criterio             | Tiempo       | Comparaciones
--------------------------------------------------------------------------------
Merge Sort      | Calificación ↓       |   0.0510 ms |    224 comps
Quick Sort      | Calificación ↓       |   0.0272 ms |    253 comps
Insertion Sort  | Calificación ↓       |   0.0501 ms |    639 comps

================================================================================
ANÁLISIS DE ORDENAMIENTO
================================================================================

Mejor algoritmo para ordenar por PRECIO: Quick Sort (0.0362 ms)
Mejor algoritmo para ordenar por CALIFICACIÓN: Quick Sort (0.0272 ms)

================================================================================
PARTE 3: Algoritmos de Búsqueda
================================================================================

 Búsqueda por ID (Búsqueda Binaria)
--------------------------------------------------

--- Búsqueda Binaria: IDs que EXISTEN ---
Resultados: 10/10 encontrados
Tiempo promedio: 0.57 µs
Comparaciones promedio: 8.6

--- Búsqueda Binaria: IDs que NO existen ---
Resultados: 0/10 encontrados
Tiempo promedio: 0.74 µs
Comparaciones promedio: 12.0
```

```
 Búsqueda por Nombre (Búsqueda Lineal - subcadena)
------------------------------------------------

--- Búsqueda Lineal: Subcadenas que EXISTEN ---
Resultados: 5/6 búsquedas con resultados
Tiempo promedio: 5.64 µs
Comparaciones promedio: 50.0

--- Búsqueda Lineal: Subcadenas que NO existen ---
Resultados: 0/6 búsquedas con resultados
Tiempo promedio: 5.84 µs
Comparaciones promedio: 50.0

================================================================================
CONCLUSIONES FINALES
================================================================================

    1. ORDENAMIENTO:
       - Quick Sort y Merge Sort son significativamente más rápidos que Insertion Sort
         para n=50, aunque la diferencia se hace más notoria con conjuntos más grandes.
       - La teoría O(n log n) vs O(n²) se confirma en la práctica.
       - Ordenar objetos complejos tiene overhead adicional por las llamadas a key_func,
         pero la complejidad algorítmica domina el rendimiento.

    2. BÚSQUEDA POR ID (Binaria):
       - La búsqueda binaria es extremadamente rápida (microsegundos) incluso con 50 elementos.
       - Es ideal para búsquedas por clave única porque requiere O(log n) comparaciones.
       - El requisito fundamental es tener los datos ordenados por el campo de búsqueda.

    3. BÚSQUEDA POR NOMBRE (Lineal):
       - La búsqueda lineal con coincidencia parcial es O(n) por cada búsqueda.
       - Es más lenta que la búsqueda binaria, pero necesaria para búsquedas textuales.
       - En producción, se usarían índices invertidos o árboles de prefijos (Trie) para optimizar.


================================================================================
PROGRAMA FINALIZADO CON ÉXITO
================================================================================
➜  parcial2-tienda-online git:(main) ✗
```

Evidencia de Repo existente: